



Internet of Things

Report

Smart Fire Fighting System

Name: Bibek Pandeya

University Id:24000863

Level: 5'A'

Table of Contents

S.N	CONTENTS
•	Introduction
1	Prototype Identification and Planning
1.1	Literature Review on Prototype Identification
1.2	Reflection on the Prototype Identification
2	System Development, developed code and planning documents for the prototype
3	Evaluation
3.1	Report on the Evaluation
•	Conclusion
4	References

Introduction

The current technique of extinguishing fire, that is, ceiling-mounted devices, is not precise. Fire extinguishing methods also tend to cause secondary flood damage that is far beyond the cost of damage due to a fire. For this reason, a “Smart Fire Fighting” system is thus proposed in this project that aims to suppress the fire source. Unlike the other systems that are fixed on the roof, my system scans through the area, locks the target, and executes intelligent suppression. My system offers a more practical, secure, and effective solution.

This project fills a critical gap in home safety. According to NFPA, many people have died due to fire in residential buildings, but very few have a working fire suppression system installed in their homes. The reason for that is, as a rule, cost considerations and fear of water damage. The available technology works well, but it is inefficient because it floods an entire room to extinguish a small fire. Concentrating the water on the fire rather than filling a whole room would prevent a secondary loss of water damage, which is often greater than the original cost of the fire.

My system was designed with two logics: low latency and minimal water use. For the latency logic, I used Arduino Nano for fast reaction and real-time implementation. Second, Water Minimization, rather than flooding the whole area, I wrote my code so that the system does not shoot water continuously; it shoots for 3 seconds and checks again if the fire is active or not.

Ultimately, I realized that remote monitoring is essential for my project. To achieve this, I implemented the Internet of Things (IoT) technology. I connected the whole system to the internet through ESP8266. Due to this, anyone will be able to see the condition of the fire and the system even if they are not in that location through the ThingsBoard dashboard.

The complete period of development, including the literature that was used in the development of architecture and the code of the intermittent algorithm of spray, was documented in this report. It concludes with the analysis of the system as per its latency, mechanical calibration, and stability of power, thus demonstrating the practicability of active suppression methods.

1: Prototype Identification and Planning

1.1 Literature Review

Before the design phase, I researched techniques, similar projects, limitations, and safety measures. In a comparison study by **Sowah et al. (2019)**, the authors classify suppression systems as ‘passive’ and ‘active’ systems. The author explains that passive systems, like smoke alarms, are not effective in cases like an empty house because the system requires a human to detect the smoke. Whereas active systems are automatic, they spray a lot of water, which can

damage the entire room. The literature demonstrates both Active and Passive systems, which influenced the design of my 'Target Active System'.

I selected the blueprint for this project after conducting some research on 'Dual-Microcontroller' systems. Research by **Sobota et al. (2016)** highlights the performance difference between single-board computers, such as Raspberry Pi, and dedicated microcontrollers. They discovered that by distributing the network responsibilities to the second microcontroller, such as ESP8266, the primary microcontroller can provide deterministic values of the Loop time. Therefore, the hardware response time, such as the water pump response time, takes less than 200 milliseconds. This research confirmed that I made the correct choice between Raspberry Pi and Arduino Nano.

Concerning the system, I will use a Monitoring system on ThingsBoard. I began to research protocols such as HTTP, MQTT, and others. **Al-Fuqaha et al. (2015)** show during the research that the time-critical systems in IoT cannot be precise with such a protocol as HTTP. They instead proposed the application of MQTT, which operates well even when the network is not stable and is quick. Based on the above results, MQTT was chosen as the communication protocol to ensure low-latency communication for critical notifications like "Fire Detected" even in scenarios where the network is unstable.

According to **Kodali & Sahu (2016)**, UART-based communication between Arduino Nano and ESP8266 enables the separation of control and networking tasks. From this reference, I understood the relation between the Wi-Fi connection and Nano. This source has particularly clarified the role of Nano when the connection on the dashboard faces problems, even if the Wi-Fi module is fully shut down, the Nano works normally, and it detects the fire and extinguishes it.

"Brain Positioning" was also one of the key design elements of my project. According to **Shi et al. (2016)**, "Edge Computing means that the processing happens as close as possible to the location of the data." In the context of my fire robot, "Edge Computing" is imperative. Situations where I relied solely on the ThingsBoard interface to command the pump to switch ON/OFF mean that the entire house could have been consumed by fire because of a mere Wi-Fi connection disruption. This gives rise to the "Single Point of Failure." I supported the argument provided in **Satyanarayana(2017)** that decision-making and critical functionalities need to occur at the point of devices themselves because devices are extremely important and should not fail because of internet connectivity before critical actions are executed. This has convinced me of the need for the "Attack Sequence Program" module within the Arduino Nano board, as the "Safety Logic" is performed at the "Edge Reasoning" level on the board itself, and the ESP8266 module is only responsible for reporting, including when the internet is down.

Moving on to designing control logic, I started researching water burst logic, as well as its alternative. According to **Tawfiq et al. (2021)**, spraying water continuously is inefficient, as, at times, the water tank will be empty when the fire is not yet fully extinguished. They proposed an

alternative named ‘Intermittent spraying Algorithm’ wherein it goes on to scan to check if the fire is active or not. This method was highly effective, and thus it was incorporated into my own project named 3-second Burst. The approach was directly instrumental in the development of the intermittent spraying logic implemented when designing the prototype.

Finally, while doing the testing, I discovered a mechanical limitation. **TowerPro Datasheet** explained that the servo motors, like SG90, have plastic gears that have gear slack of some degrees. This research helped diagnose why there is a false servo angle on the dashboard.

1.2 Reflection

Now, I am using Nano and ESP8266 to create my IoT project. I realized that although the Raspberry Pi is powerful, the fact that it is a fully-fledged computer makes it unnecessarily complex to perform the given job. It requires complex configuration and has a large bootup time. As my project involves fire detection, the hardware must react instantly. From my research on real-time systems, I learned that its operating system causes latency to exist, which I realized was not acceptable for an instantaneous fire suppression system. That is the reason for choosing the nano as the primary device for my project; it turns on instantly, controls servo motors perfectly, commands the motor to move to the exact location when fire is detected, and reacts instantly when needed. Moreover, the reason for selecting the ESP8266 to address the connectivity problem: Nano did not have Wi-Fi built in, so the combination of the ESP8266 addresses Nano’s lack of native connectivity. It is a cheap and reliable way to enable the nano to transmit telemetry data to the internet and the dashboard. The adoption of UART communications was very informative; serialization was hard to implement between the two devices at first. However, I was able to interpret how the sensor information is supposed to be sent. I focused on a Dual microcontroller approach using the Nano and ESP8266 rather than using the Raspberry Pi. This project was a confirmation for me of the ‘Dual Microcontroller’ methodology, as it showed that it is far safer to divide the ‘muscle’ computer (Nano) from the ‘voice’ computer (ESP8266). Lastly, the calibration process taught me that software logic should always take into consideration physical hardware constraints, including gear slack.

2: Development

The system implementation phase is now concerned with the integration of the mechanically assembled and the control logic.

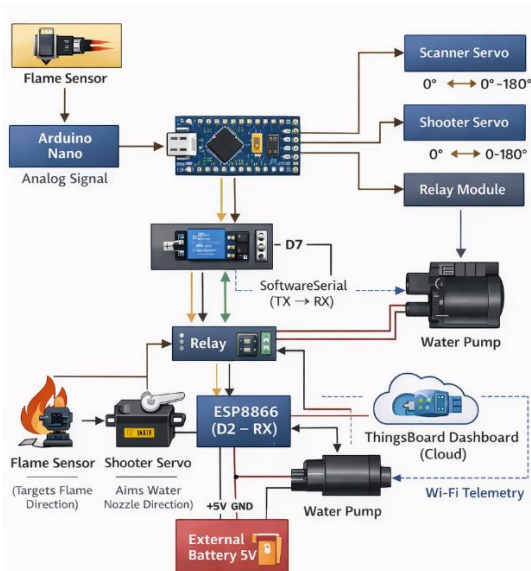


Figure 1 System Block Diagram of logic data flow.

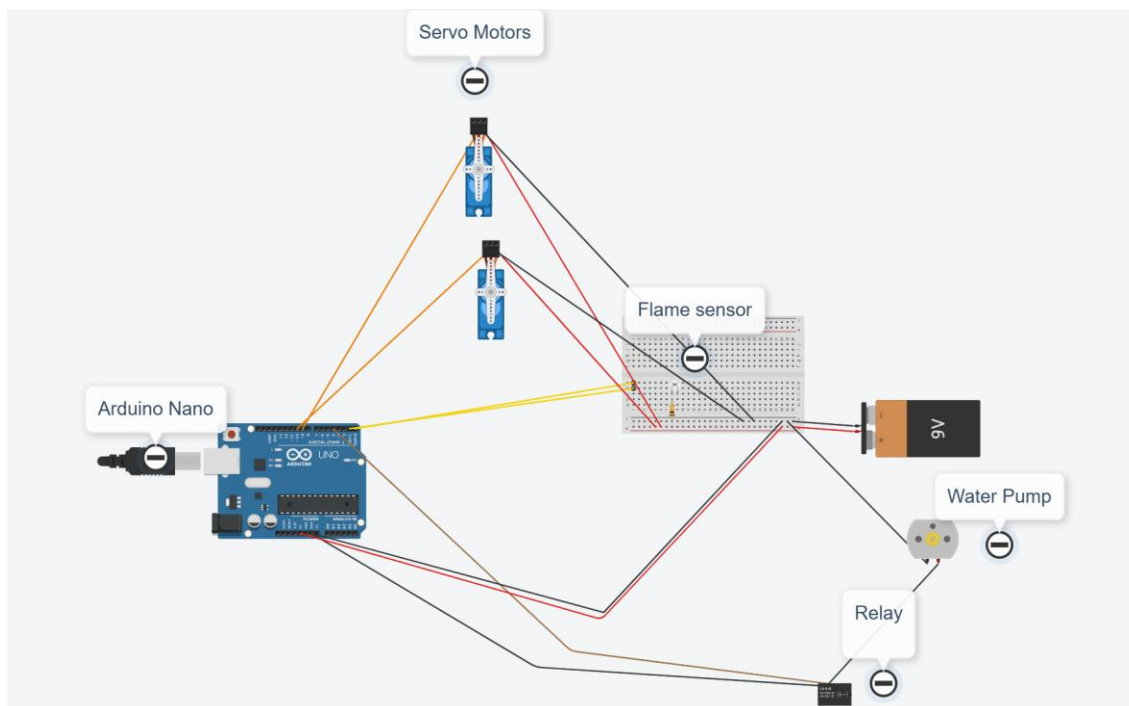


Figure 2: Circuit Schematic Diagram Showing the split-power architecture.

Firstly, I worked on the configuration of servo motors. There are two servo motors used in this specific project. According to the control logic, the main servo scans, and the secondary servo aims for the coordinates pointed out by the flame sensor in order to perform suppression actions. After working on the logic, I first tested the motors using the nano, as they worked perfectly. The flame sensor was mounted above the first servo so that it would be able to scan the fire at the exact location. The calibration procedure was established, where the angles of the servo were shifted to offset the actual nozzle axis. The first servo rotates from 0 degrees to 180 degrees to scan the fire. Similarly, to have a favorable attack angle and provide the nozzle trajectory, the targeting servo was mounted with a vertical offset to the scanning servo. So, after the fire sensors detect the fire in X direction, the microcontroller records the angular coordinate of the first motor facing, then it issues a command to the secondary servo to move to the exact location where the first one is facing.

```
// --- SETTINGS ---  
const int fireThreshold = 200;  
const int scanSpeed = 60;
```

Figure 3: Fire Threshold Logic

Now, for the logic of the fire sensor, I set the value to if it's above 200, it is not fire, and if it drops below 200, it is fire. So, the implementation is the servo motor with an attached flame sensor that rotates till the flame value is above 200, but when it drops below that value, it enters "AttackSequence" mode, which means the first servo stops second servo aims, then the water pump starts and sprays water to extinguish the fire. The IR Flame Sensor works with reverse logic, meaning the stronger the infrared light is, the lower the resistance, thus a small analog voltage reading would result (0-1023).

```

void attackSequence(int detectedAngle) {
  // 1. Initial Alert
  Serial.println("fire:1");
  Serial.println("pump:0");

  // 2. STOP SCANNING & LOCK TARGET
  int bestAngle = detectedAngle;
  int lowestVal = 1023;

  int startSearch = detectedAngle - 15;
  int endSearch = detectedAngle + 15;
  if (startSearch < 0) startSearch = 0;
  if (endSearch > 180) endSearch = 180;

  for (int i = startSearch; i <= endSearch; i += 3) {
    scanner.write(i);
    delay(30);
    int val = analogRead(flamePin);
    if (val < lowestVal) {
      lowestVal = val;
      bestAngle = i;
    }
  }
}

```

Figure 4: Attacksequence Logic

Arduino IDE was used to develop firmware. The Servo library was used for driving the motor, and PubSubClient for MQTT communications. The logic of the initial code is that as soon as the system detects a fire, it immediately sends a signal ‘fire:1’ to the dashboard. The most important development in the working of the system was to figure out a reliable data protocol. To structure the data being transmitted in a readable manner, I had written the code in such a way that it used a series of keys and values: angle: 90, pump:1. Conversely, the code was supposed to interpret such particular strings at the ESP 8266 and then serialize them in JSON format, which could be read by ThingsBoard.

```

// Listen to Nano
if (nanoSerial.available()) {
  String data = nanoSerial.readStringUntil('\n');
  data.trim();

  if (data.length() > 0) {
    Serial.print("Received: ");
    Serial.println(data);

    // Parse "key:value"
    int separatorIndex = data.indexOf(':');
    if (separatorIndex != -1) {
      String key = data.substring(0, separatorIndex);
      String valueStr = data.substring(separatorIndex + 1);

      // Prepare JSON payload
      StaticJsonDocument<200> doc;

      // Map Nano keys to ThingsBoard keys
      if (key == "fire") doc["fire_status"] = (valueStr.toInt() == 1);
      else if (key == "pump") doc["pump_status"] = (valueStr.toInt() == 1);
      else if (key == "angle") doc["servo_angle"] = valueStr.toInt();
      else if (key == "water") doc["water_flow"] = valueStr.toFloat();

      // Convert JSON to String
      char payload[256];
      serializeJson(doc, payload);

      // Send to ThingsBoard
      client.publish("v1/devices/me/telemetry", payload);
    }
  }
}

```

Figure 5: Data and Commands Handling Logic

Due to this logic, the observer receives the alert quickly, even before the system begins to work. Now, for the last few lines of code ‘Loop’, the logic is, the code checks the flame sensor data every 3 degrees. It remembers the lowest value or the strongest fire intensity. At last, the variable ‘bestangle’ takes the exact center of the fire to pump the water in the proper direction.

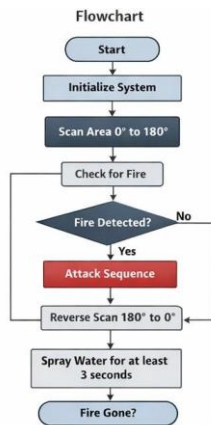


Figure 6: Flowchart of Fire Detection and Attack Sequence logic.

Now, for the power supply distribution, I chose a split-power system because, during testing, Nano was unable to provide the 5V for the high-torque servo motors and the water supply. So, to solve the problem, I used an external high-voltage battery for servo motors and a separate battery and relay module. Nano sends a signal to my relay module, which switches the battery power to the pump. This protects the sensitive microcontroller.

Moreover, in the case of the water pump or extinguishing method, I put in place a program of smart bursts. This is a waste of water as the traditional systems spray water quickly until they are turned off manually. To address the issue, I therefore added a 3-second delay in my code. In my reasoning, on commencement of the pump, the first thing is to pump water for 3 seconds, and then the pump gets into a loop to examine whether the fire sensor has given a signal on the fire status. In the event of the status not being active and normal, the pump goes to standby.

To complete the last stage of the Visualization of the system, I selected ThingsBoard. The main aim was to view the conditions using my laptop and not to get close to the fire. The protocol that I employed was MQTT, which transmits the data sent by the ESP8266 to the dashboard or cloud.

I was able to arrange the layout in two simple rows. Under the first row, in the section of Status Alert, I added the System status. And ‘pump status’ widgets. Within the system, it is possible to see whether the fire is running or not. On the same note, when it comes to the pump status, the widget functionality would be easy to verify whether the pump is on or off after the fire has been detected.

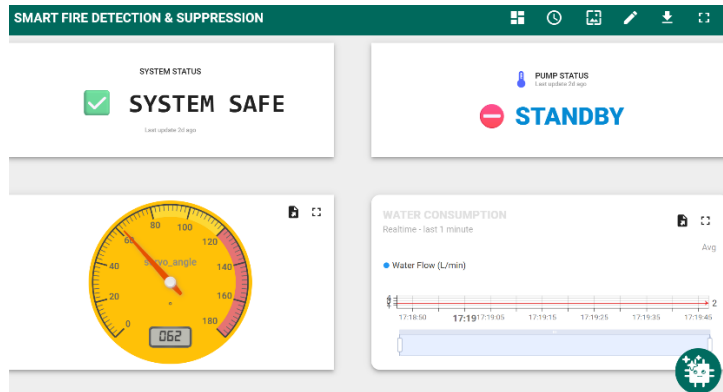


Figure 7: ThingsBoard Dashboard Visualization.

The second row, i.e., Live Data, is now added with the Servo Gauge and Water Consumption chart. In the first widget, I added the angle of the first servo motor, and in the Water Consumption chart, the logic of a 3-second reset.

3: Evaluation

3.1 Report on the Evaluation

To test it, I did this activity inside. This test was aimed at checking the correctness of the system, its functionality with the servo motors, and its speed in data transfer. To consider this activity in more detail, I separated it into different cases:

This test aimed to measure the latency of the system between igniting the fire and pump ON. Five runs were carried out using a stopwatch to estimate the average response time.

Table: System Response and Latency Test

Test No.	Distance	Response Time	Result
1	10cm	150ms	Success
2	10cm	145ms	Success
3	20cm	180ms	Success
4	20cm	210ms	Success
5	30cm	290ms	Success

Average - 195ms

From the table, the average response time of the system was approximately 195m. This is significant because it barely makes the 200-ms cut-off recommended by the research of **Sobota et al. (2016)** that I read earlier, demonstrating that Nano is indeed processing the signal in real-time. I did observe that there was a small anomaly at 30cm in terms of the response. Such an

anomaly suggests a decrease in flame sensor accuracy over long distances, which leads to a minor signal processing lag.

The next one is 'Dashboard Connectivity & Latency'. In this test, initially, I looked at the speed between the physical status and the dashboard. I placed my laptop near the setup of my project, then triggered the flame sensor to analyze how much time it would take to display 'Fire Detected.' Symbol in the dashboard. I noticed that there was a minimal delay between the physical world updates and the updates shown on the dashboard. The minimal delay is because the data is routed through Nano > Esp8266 > Router > Cloud > laptop browser. Finally, even though there is a small gap, the dashboard visualization was clear, and the connection was stable, which indicates that the MQTT protocol was perfect.

Another test is 'Mechanical Accuracy and the Servo Angle Discrepancy'. This was the most important part of the testing process. During the run, I noticed that there was a technical issue regarding the angle of the dashboard gauge and the actual angle of the servo motor. The problem was in the dashboard; the angle of the servo was 117 degrees, and compared to the physical system, I observed a slight difference; it was about 105-110 degrees. I traced this back to two problems. First one 'Mechanical slack'. The difference was explained by the mechanical backlash of SG90 servo motors, which have plastic gears with reduced tolerance. Basically, due to the physical slack, the servo will settle at 105-110 degrees. The second one, Telemetry Latency. Servo motors scan and move at very high speed, so the position data was sent off a few seconds earlier to the cloud. Overall, the little lag did not affect my overall system.

The last test I did was the Power Stability test. So, for this test, I analyzed my logic of split-power supply design. On the first stage of development, Instability in voltage during testing, the servo was moving too slowly, and whenever I switched on the pump system, it would always restart. However, during the final analysis, everything worked properly without any problem with the help of an external battery and relay module.

Overall, the system satisfies every objective. The system detects fire instantly and works with remote monitoring. The problem with the display of the angle of the servo motor is negligible; it did not affect the system. Even if the prototype is going into the real world, the industries would use a metal gear instead of a plastic gear.

Conclusion

In conclusion, I have completed the required tasks for the whole exercise, that is, the development of a working firefighting robot that has a Wi-Fi module. I successfully integrated with the hardware part and the Wi-Fi part, establishing a system that is fast and intelligent. All my logic worked perfectly, like identifying the fire source, targeting motors, and conserving the water.

I found a serious problem when testing the communication between Arduino Nano and ESP8266. I found that the system had a problem of mismatched Baud Rates, as the Nano sent data in 9600-baud while the ESP8266 was set in 115200-baud mode. After setting the same baud rate, the dashboard starts working.

In terms of looking towards real-world implementation, I would implement two basic changes. First, there is a need for advancement in sensor technology. Although my program detects fire accurately according to a basic analog threshold test (anything lower than 200), a basic light sensor is prone to deception by sunlight, as well as being blinded by smoke. To implement real use, the need for a thermal camera cannot be overemphasized. Second, I could implement Machine Learning techniques for the basic threshold test. Although a basic if-else test cannot distinguish between a heater and an actual fire, a basic AI program within a thermal camera could implement such a test easily.

Lastly, this project has proved the value of the research project called “Edge Computing” that I undertook at the beginning. The idea of implementing the safety functions hardcoded in the Nano, allowing the robot to continue functioning even without an internet connection, ensures that it is immune to the “Single Point of Failure” commonly experienced by Consumer IoT products. The lesson from this module, in the context of IoT, is realizing that simply being connected to the internet is but one aspect, and there is always a middle ground in terms of processing locally and being remotely accessed.

Thank You

References

- Al-Fuqaha, A., Guizani, M., Mohammadi, M., Aledhari, M., & Ayyash, M. (2015). Internet of Things: A survey on enabling technologies, protocols, and applications. *IEEE Communications Surveys & Tutorials*, 17(4), 2347–2376. <https://doi.org/10.1109/COMST.2015.2444095>
- Kodali, R. K., & Sahu, S. (2016). An IoT-based smart security system. *2016 International Conference on Control, Instrumentation, Communication and Computational Technologies (ICCICCT)*, 582–585. IEEE. <https://doi.org/10.1109/ICCICCT.2016.7988014>
- Satyanarayanan, M. (2017). The emergence of edge computing. *Computer*, 50(1), 30–39. <https://doi.org/10.1109/MC.2017.9>

- Shi, W., Cao, J., Zhang, Q., Li, Y., & Xu, L. (2016). Edge computing: Vision and challenges. *IEEE Internet of Things Journal*, 3(5), 637–646. <https://doi.org/10.1109/JIOT.2016.2579198>
- Sobota, J., Piguet, S., & Ricker, G. (2016). Raspberry Pi vs. Arduino: A comparison of real-time control performance. *IFAC-PapersOnLine*, 49(6), 273–278. <https://doi.org/10.1016/j.ifacol.2016.07.127>
- Sowah, R. A., Ofoli, A. R., Krakani, S., & Fiawoo, S. (2019). Hardware module design of a real-time multi-sensor fire detection and notification system using fuzzy logic. *International Journal of Engineering Research in Africa*, 41, 126–141. <https://doi.org/10.4028/www.scientific.net/JERA.41.126>
- Tawfiq, M., Al-Sarray, B., & Al-Obaidi, H. (2021). Design and implementation of a smart firefighting robot using IoT. *IOP Conference Series: Materials Science and Engineering*, 1090(1), Article 012015. <https://doi.org/10.1088/1757-899X/1090/1/012015>
- TowerPro. (n.d.). *SG90 micro servo data sheet*. Retrieved January 5, 2026, from <http://www.towerpro.com.tw>